CSE323 Term Project: The Full Engineering Cycle

Goal: Design, build, and validate an automated sub-system (e.g., Customer Ordering, Kitchen Display, or Management Analytics) by prioritizing architectural integrity over mere code generation.

Phase 1: Requirement Discovery & Traceability

- **Actor Classification**: Define the Primary (initiating), Supporting (secondary), and Offstage actors for your system.
- **Traceability Heatmap**: Construct a matrix to ensure no "orphaned" requirements exist and every feature is mathematically justified.
- **Persona Discovery**: Use an "AI User" avatar to act as a frustrated or malicious student to uncover at least 5 "hidden" requirements or edge cases.

Phase 2: Design & Specification

- **Gherkin Scripting**: Translate all core user stories into structured Gherkin syntax (Given/When/Then).
- **The Refinement Loop**: Conduct a "Senior QA Audit" to eliminate unquantifiable adjectives like "fast" or "secure" and replace them with measurable technical metrics.
- **UML Modeling**: Document the system flow using **System Sequence Diagrams** (for happy/failure paths) and **Activity Diagrams** that integrate code decision points.
- **Information Hiding**: Design your API contracts such that teams/AI only need to respect shared interfaces, keeping internal stack logic hidden.

Phase 3: Test-Driven Implementation

- **Test-Driven Prompting (TDP)**:
    1. **The Failing Test**: Write a unit test first to establish a "mathematical boundary".
    2. **The Edge Case Cage**: Use "Padlocks" (boundary, threshold, and extreme constraints) to block the AI from hallucinating logic.
    3. **Iteration**: Prompt the AI until the logic perfectly fits the boundary established by the failing test.
- **Vertical Slicing**: Divide labor by sub-problem (UI/Logic/DB in one stack) to ensure localized context and failure resilience.

Phase 4: Validation & Pipeline Engineering

- **The Testing Pyramid**: Implement a suite following the 70% Unit, 20% Integration, and 10% System/E2E ratio.
- **Automated Validation**: Convert your Gherkin scenarios into executable **Playwright** scripts using the Page Object Model.
- **Final Validation**: Document how your software not only "works" (Verification) but "solves the right problem" (Validation).